

程序设计实习大作业报告

一、程序功能介绍

本程序主要由两个板块构成：

- 1、代码可视化和分析板块：用户可以输入代码，进而逐步运行并展示代码的内存结构，帮助用户更好理解 cpp 的底层原理；另外，还设计了 ai 辅助功能，ai 会自动分析输入的代码，也支持进一步的追问；
- 2、知识库板块：用户可以在这里创建自己的错题本，也可以收入笔记，从而形成个人知识库，有效帮助个人学习

二、小组成员分工

- a) 张宇轩：代码解析以及 ai 辅助功能
- b) 吴凡：UI 设计
- c) 陈宣宇：知识库功能

三、项目各模块和类设计细节

- a) 代码解析及 ai 辅助：

考虑到大作业的属性，该模块没有使用现有的工具，而是自己写了一个简单的代码解析器，总的来说， workflow 如下：

源代码字符串→

[1]Lexer::tokenize()→

Token 序列(vector<Token>)→

[2]Parser::parse()→

AST 语法树(Program 根节点)→

[3]ASTAnalyzer(静态分析)→

标注后的 AST (Binding、SymbolTable、ClassModel) →

[4]ExecutorVisitor(动态执行)→

Memory + 执行轨迹(vector<Stepsnapshot>)→

[5]前端查询→

CodeAnalyzer 通过 getExecutionTrace() / getAllClassViews()等接口暴露数据

至于 ai 辅助功能, 则主要通过 ai_analyzer 实现功能, 仍然由

code_analyzer 暴露接口, 关键就是解析 json 字符串然后设计好 prompt

下面是部分比较重要的类设计:

1、parser:

利用 parser, 采用自顶向下的方式构建出 AST 语法树

2、binding/symbol_table/memory:

这是一个比较 tricky 的地方, 就是作用域的问题, 这里使用了 binding

这个关键的数据结构, 利用槽和 scope 维护词法作用域; 然后在

memory 当中再动态维护 lexicalFrame / lexicalNameFrames /

lexicalTypeFrame 保证绑定正确; 此外还使用 lexicalhistory 确保显示的数据正确

3、visitor:

使用访问者设计模式; 分离静态/动态, 在一定程度上也模仿了 cpp 的运行

b) UI 设计:

代码执行可视化模块说明

文件: codeeditor.h/.cpp、mainwindow.h/.cpp

说明: 基于 Qt6 的 Qwidget, QPlainTextEdit, QPushButton 等组件, 搭建了代码编辑器, 可视化面板, ai 侧边栏, 实现了代码运行的栈帧, 对象, 类的可视化

一、CodeEditor (代码编辑器)

继承 QPlainTextEdit, 提供:

- 行号区 (LineNumberArea): 左侧显示行号, 随滚动同步
- 执行标记:

setCurrentLine(n) → 当前行左侧红色箭头

setNextLine(n) → 下一行左侧绿色箭头

clearExecutionLines() → 清除所有箭头

二、MainWindow 可视化面板

整体三栏布局: 代码编辑器 | 可视化面板 | AI 侧边栏

右侧面板 (QScrollArea, 从上到下):

1. Event Info → 当前事件名、行号、函数名
2. Stack Frames → 每帧一个蓝色 GroupBox, 内含变量表

(Name/Type/Value)

3. Stack Objects → 黄色 GroupBox, 栈上对象的成员表格、基类、虚表

4. Heap Objects → 橙色 GroupBox, 堆上对象的成员表格、基类、虚

表

5. Class Definitions → 类名、继承深度、成员/方法、虚表

数据流:

onStartClicked → loadCode → runFullAnalysis →

getExecutionTrace

onStepChanged(step) → 从 m_trace[step] 取快照 → 依次调用 5 个

render*()

→ updateExecutionArrows() → updatePointerArrows()

三、指针箭头绘制 (PointerArrowOverlay)

透明 QWidget 覆盖于可视化面板上方, 用 QPainter 绘制贝塞尔曲线箭

头:

- 起点: 变量 Name 格中心
- 终点: 目标对象 GroupBox 标题左侧
- 颜色: 蓝色(→Heap)、琥珀色(→Stack)
- 曲线向左弯折, 避免穿越文字
- 三级更新机制: layout.activate() + Resize 事件 + singleShot(0) 兜底

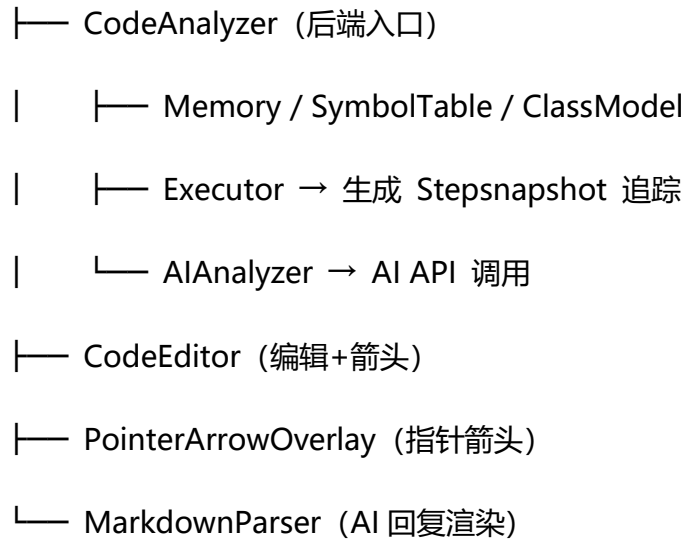
四、AI 侧边栏

- Markdown 渲染 (复用 MarkdownParser), 支持表格/代码块
- 聊天气泡: 用户蓝色右对齐, AI 绿色左对齐
- 异步调用: QtConcurrent::run + QFutureWatcher, 不阻塞 UI

- 容错：代码有语法错误时绕过 loadCode 直接提交 AI

五、依赖链

MainWindow



c) 知识库：

模块说明：

本模块包含知识库和错题本两大核心功能。支持用户以多级分类整理

markdown 格式的笔记，可以插入图片，并提供 markdown 渲染；支持搜

索知识点和错题；目录部分支持多种右键操作；其中错题本部分还可以管理

每个错题的关联知识点，并统计推荐薄弱知识点。

类设计：

核心类：KnowledgeBookWidget

继承自 QDialog，作为模块顶层窗口，管理知识库与错题本的界面布局、树

形目录、文件读写、搜索等全部交互逻辑。

关键成员：

·QStandardItemModel *kbModel / *errorModel: 分别维护知识库与错题本的树形数据结构。

·QTreeView、QTextEdit 等 UI 控件: 提供目录浏览与 Markdown 编辑/预览界面。

·QList<QPair<QString, QString>> allKnowledgeItems / allErrorItems: 缓存所有节点路径, 供搜索与统计使用。

关键方法:

·scanFolder(): 递归扫描本地目录, 构建树形模型 (支持无限嵌套分类)。

·onTreeWidgetItemClicked() / onErrorTreeWidgetItemClicked(): 响应目录点击, 加载文件内容并渲染。

·onSaveClicked() / onErrorSaveClicked(): 保存编辑内容 (错题本额外处理 YAML 元数据)。

·右键菜单相关: createCategory()、renameItem()、moveItem()、deleteItem() 等, 实现节点增删改移。

·搜索相关: onSearchButtonClicked() 等, 弹出搜索面板, 按名称或内容检索, 点击结果自动定位。

·onStatsButtonClicked(): 统计错题关联知识点, 展示 Top 3 高频薄弱知识点。

·插入/清理图片相关: onInsertImageClicked() / onCleanImagesClicked(), 插入本地图片 (自动重命名防冲突) 与清理多余图片。

·closeEvent(): 在关闭窗口时检查未保存内容, 弹出保存提示。

辅助类：MarkdownParser

负责将 Markdown 文本转换为 HTML，供 QTextEdit 渲染。

关键方法：

·toHtml(markdown, baseDir)：解析标题、加粗、斜体、删除线、行内代码、代码块、有序/无序列表、引用、表格、水平线、图片等语法。

四、项目总结与反思

整个项目显然还是不够成熟，如果想要获得比较好/完善的体验，那么直接使用 monaco/tree-sitter 等工具应该是更好的。当然，自己在构建这种比较复杂的项目时，也收获了很多经验和教训；在和组员们的交流沟通中，收获了成长，我想这是更为重要的。总之感谢组员们、感谢老师、感谢程设，再见！